



Chapter Five

Basic Computer Organization and Design

Outline

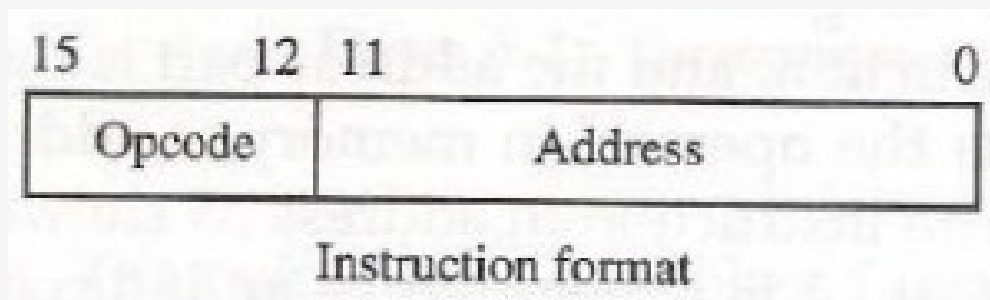
- Instructional Code
- Computer Registers
- Computer Instructions
- Timing and Control
- Memory Reference Instructions
- Design of Basic Computer
- Design of accumulator Logic

Instruction Codes:

- ❖ The organization of the computer is defined by its internal registers, timing and control structure, and the set of instructions that it uses.
- ❖ Internal organization of a computer is also defined by the sequence of micro-operations it performs
- ❖ It is possible a Computer can be instructed about the specific sequence of operations it must perform by means of a Program.
- ❖ *Program*: set of instructions that specify the operations, operands, and the sequence by which processing has to occur.

Cont..

- ❖ **Instruction**: a binary code that specifies a sequence of micro-operations for the computer.
- ❖ The computer reads each instruction from memory and places it in a control register.
- ❖ **Instruction Code**: group of bits that instruct the computer to perform specific operation.
- ❖ Instruction code is usually divided into two parts: Opcode and address(operand)



Cont..

Operation Code (opcode):

- ✓ group of bits that define the operation
- ✓ Eg: add, subtract, multiply, shift, complement.
- ✓ No. of bits required for opcode depends on no. of operations available in computer.
- ✓ n bit opcode $\geq 2^n$ (or less) operations

Address (operand):

- ✓ specifies the location of operands (registers or memory words)
- ✓ Memory words are specified by their address
- ✓ Registers are specified by their k -bit binary code
- ✓ k -bit address $\geq 2^k$ registers

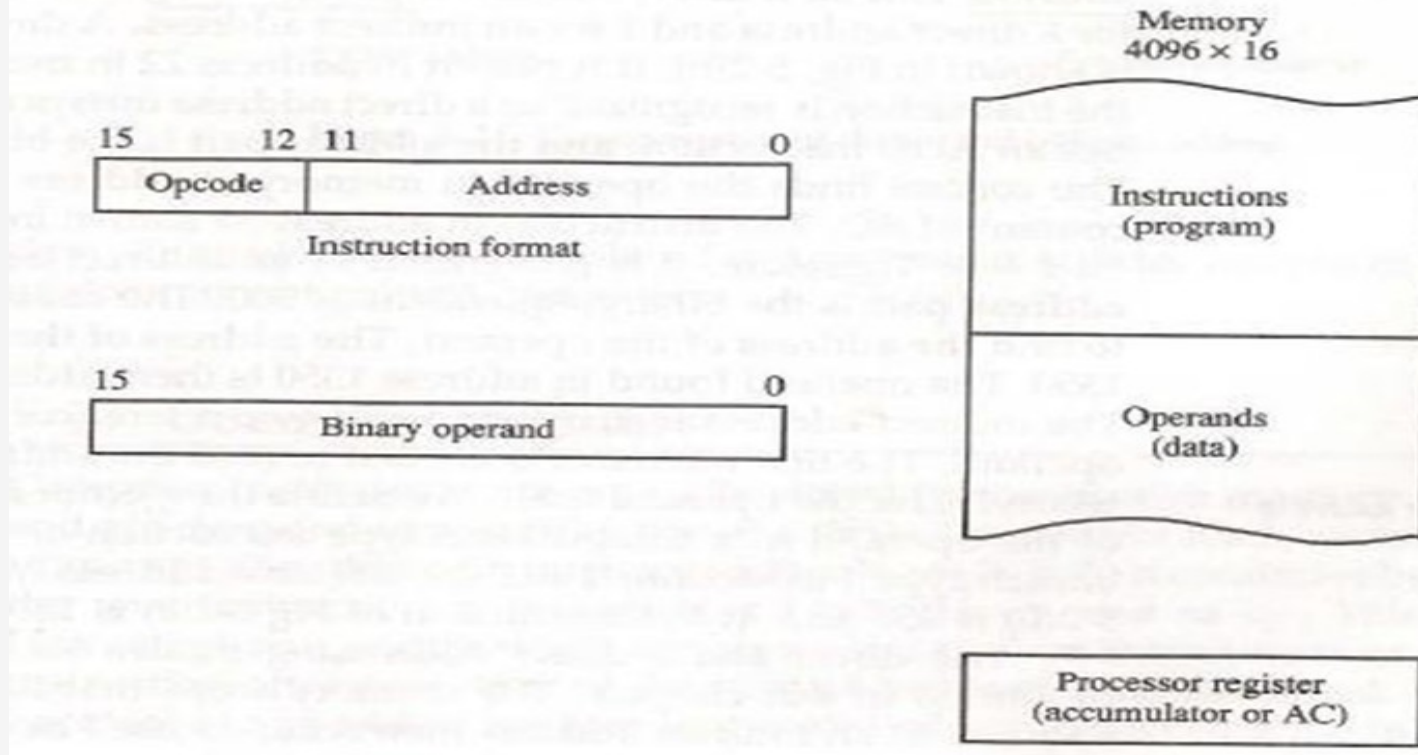
Cont..

Stored Program Organization:

- The ability *to store and execute instructions* is the most important property of a general-purpose computer.
- That type of stored program concept is called stored program organization.
- The simplest way to organize a computer is to have one processor register and an instruction code format with two parts .
- The below figure shows the stored program organization

Cont..

Figure 5-1 Stored program organization.



Cont..

- Instructions are stored in one section of memory and data in another.
- For a memory unit with 4096 words we need 12 bits to specify an address since $2^{12} = 4096$.
- If we store each instruction code in one 16-bit memory word, we have available four bits for the operation code (abbreviated opcode) to specify one out of 16 possible operations, and *12 bits to specify the address of an operand.*

Cont..

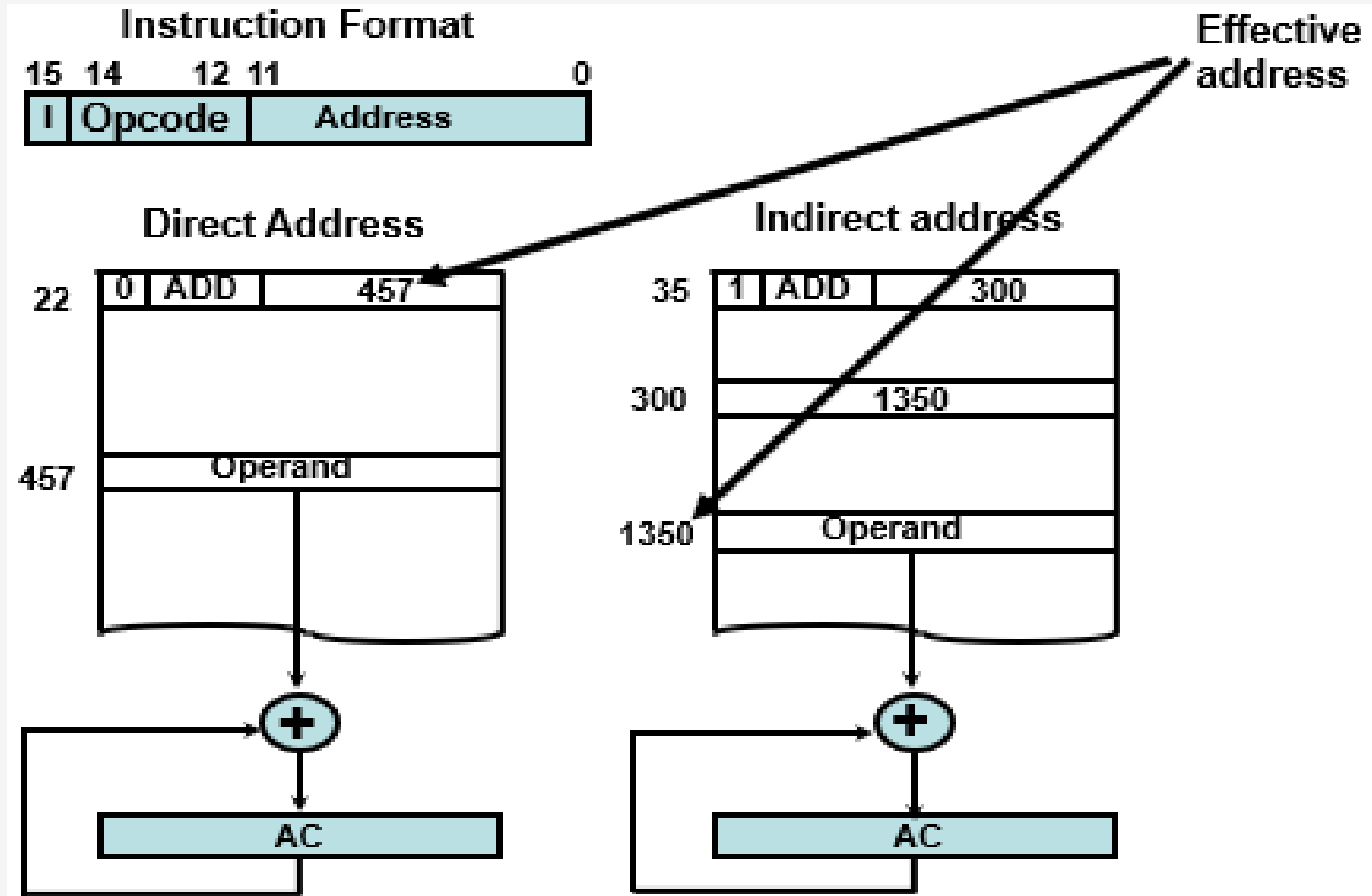
➤ ***Accumulator (AC):***

- ✓ Computers that have a single-processor register usually assign to it the name accumulator and label it AC.
- ✓ The operation is performed with the memory operand and the content of AC.

Addressing of Operand:

- Sometimes convenient to **use the address bits** of an instruction code not as an address but as the actual operand.
- When the second part of an instruction ***code specifies an operand***, the instruction is said to have an ***immediate operand***.
- When the second part ***specifies the address of an operand***, the instruction is said to have a ***direct address***.

Cont..



Cont..

A direct address instruction

- It is placed in address 22 in memory.
- The I bit is 0, so the instruction is recognized as a direct address instruction.
- The opcode specifies an ADD instruction, and the address part is the binary equivalent of 457.
- The control finds the operand in memory at address 457 and adds it to the content of AC.

indirect address instruction

- has a mode bit $I = 1$.
- The address part is the binary equivalent of 300.
- The control goes to address 300 to find the address of the operand.
- The address of the operand in this case is 1350.
- The operand found in address 1350 is then added to the content of AC.

Computer Registers

What is the need for computer registers?

- ❖ Instruction sequencing needs a counter to calculate the address of the next instruction after execution of the current instruction is completed (PC).

Types of register

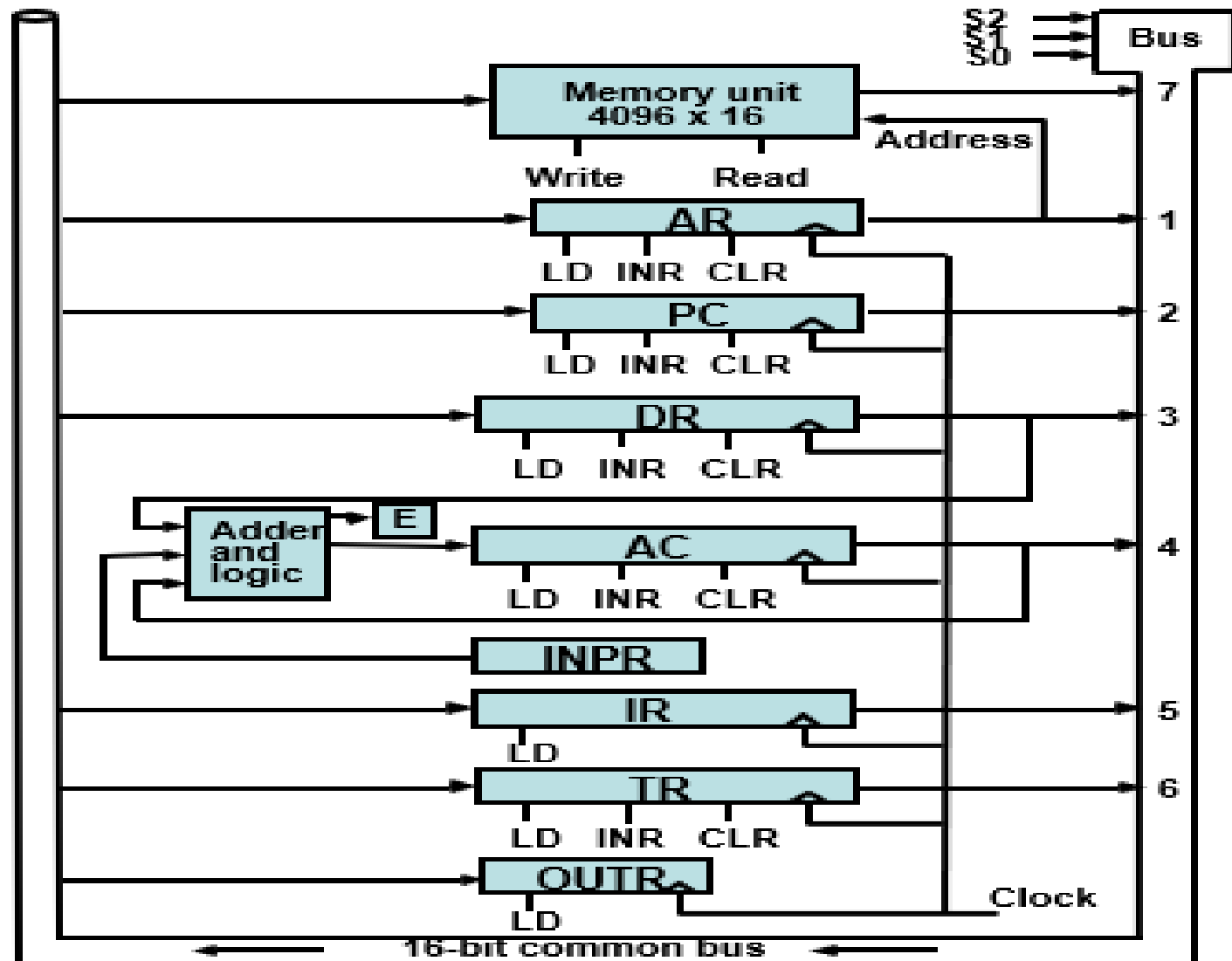
TABLE 5-1 List of Registers for the Basic Computer

Register symbol	Number of bits	Register name	Function
<i>DR</i>	16	Data register	Holds memory operand
<i>AR</i>	12	Address register	Holds address for memory
<i>AC</i>	16	Accumulator	Processor register
<i>IR</i>	16	Instruction register	Holds instruction code
<i>PC</i>	12	Program counter	Holds address of instruction
<i>TR</i>	16	Temporary register	Holds temporary data
<i>INPR</i>	8	Input register	Holds input character
<i>OUTR</i>	8	Output register	Holds output character

Common Bus System:

- The basic computer has eight registers, a memory unit, and a control unit
- Paths must be provided to transfer information from one register to another and between memory and registers.
- A more efficient scheme for transferring information in a system with many registers is to use a common bus.
- The connection of the registers and memory of the basic computer to a common bus system is shown in Fig. 5-4.

Cont..



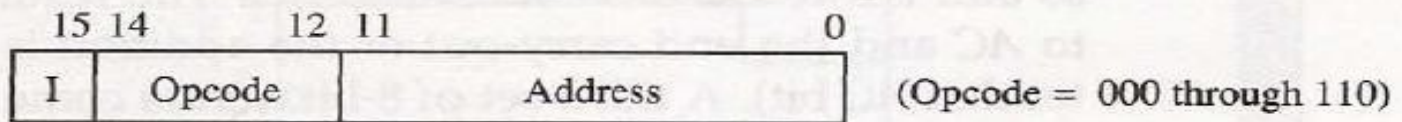
Cont..

- The outputs of seven registers and memory are connected to the common bus.
- The specific output that is selected for the bus lines at any given time is determined from the binary value of the selection variables **S₂, S₁, and S₀**.
- The number along each output shows the decimal equivalent of the required binary selection.
- For example, the number along the output of DR is 3. The 16-bit outputs of DR are placed on the bus lines when $S_2S_1S_0 = 011$.

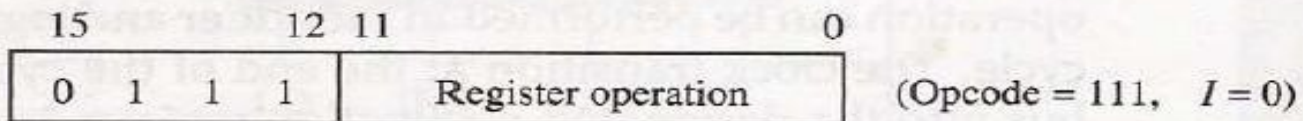
Computer Instructions:

- The basic computer has three instruction code formats, as shown in Fig. 5-5. Each format has 16 bits.

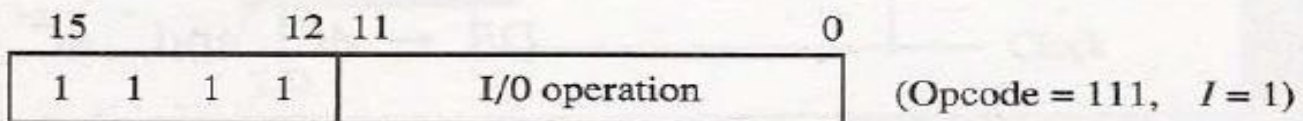
Figure 5-5 Basic computer instruction formats.



(a) Memory – reference instruction



(b) Register – reference instruction



(c) Input – output instruction

Cont..

- ❖ The operation code (opcode) part of the instruction contains three bits and the meaning of the remaining 13 bits depends on the operation code encountered.
- ❖ **A memory-reference instruction** uses 12 bits to specify an address and one bit to specify the addressing mode I.
- ❖ I is equal to 0 for direct address and to 1 for indirect address.

Cont..

- ❖ *A register-reference instruction* specifies an operation on the AC register. So an operand from memory is not needed. Therefore, the other 12 bits are used to specify the operation to be executed.
- ❖ *An input—output instruction* does not need a reference to memory and is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction.
- ❖ The remaining 12 bits are used to specify the type of input—output operation.
- ❖ The instructions for the computer are listed in Table 5-2.

Timing and Control:

- The timing for all registers in the basic computer is controlled by a *master clock generator*.
- The clock pulses are applied to all flip-flops and registers in the system, including the flip-flops and registers in the control unit.
- The clock pulses do not change the state of a register unless the register is enabled by a control signal.
- The control signals are generated in the control unit and provide control inputs for the multiplexers in the common bus, control inputs in processor registers, and microoperations for the accumulator.
- There are two major types of control organization:
 - *Hardwired control*
 - *Microprogrammed control*

Cont..

❖ The differences between hardwired and microprogrammed control are

Hardwired control	Microprogrammed control
✓ The control logic is implemented with gates, flip-flops, decoders, and other digital circuits.	✓ The control information is stored in a control memory. The control memory is programmed to initiate the required sequence of microoperations.
✓ The advantage that it can be optimized to produce a fast mode of operation.	✓ Compared with the hardwired control operation is slow.
✓ Requires changes in the wiring among the various components if the design has to be modified or changed.	✓ Required changes or modifications can be done by updating the microprogram in control memory.

Memory-Reference Instructions...

Table 5-4 lists the seven memory-reference instructions.

TABLE 5-4 Memory-Reference Instructions		
Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], \quad E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

Cont..

- ❖ The decoded output D_i for $i = 0, 1, 2, 3, 4, 5$, and 6 from the operation decoder that belongs to each instruction is included in the table.
- ❖ *The effective address of the instruction is in the address register AR* and was placed there during timing signal **T2** when $I = 0$, or during timing signal **T3** when $I = 1$.
- ❖ The execution of *the memory-reference instructions* starts with timing signal **T4**.

Cont..

AND to AC:

- ❖ This is an instruction that performs the AND logic operation on pairs of bits in AC and the memory word specified by the effective address.
- ❖ The result of the operation is transferred to AC.
- ❖ The microoperations that execute this instruction are:

$D_0T_4: DR \leftarrow M[AR]$
 $D_0T_5: AC \leftarrow AC \wedge DR, SC \leftarrow 0$

ADD to AC:

- ❖ This instruction adds the content of the memory word specified by the effective address to the value of AC.
- ❖ The sum is transferred into AC and the output carry Cout is transferred to the E (extended accumulator) flip-flop.
- ❖ The microoperations needed to execute this instruction are

$D_1T_4: DR \leftarrow M[AR]$
 $D_1T_5: AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$

Cont..

LDA: Load to AC

- ❖ This instruction transfers the memory word specified by the effective address to AC.
- ❖ The microoperations needed to execute this instruction are

$D_2T_4: DR \leftarrow M[AR]$
 $D_2T_5: AC \leftarrow DR, SC \leftarrow 0$

STA: Store AC

- ❖ This instruction stores the content of AC into the memory word specified by the effective address
- ❖ Since the output of AC is applied to the bus and the data input of memory is connected to the bus, we can execute this instruction with one microoperation.

$D_3T_4: M[AR] \leftarrow AC, SC \leftarrow 0$

Cont..

BUN: Branch Unconditionally

- ❖ This instruction transfers the program to the instruction specified by the effective address.
- ❖ The BUN instruction allows the programmer to specify an instruction out of sequence and we say that the program branches (or jumps) unconditionally.
- ❖ The instruction is executed with one microoperation:

$$D_4T_4: PC \leftarrow AR, SC \leftarrow 0$$

Cont..

BSA: Branch and Save Return Address

- This instruction is useful for branching to a portion of the program called a subroutine or procedure.
- When executed, the BSA instruction stores the address of the next instruction in sequence (which is available in PC) into a memory location specified by the effective address.
- The effective address plus one is then transferred to *PC* to serve as the address of the first instruction in the subroutine.
- This operation was specified with the following register transfer:

$$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$$

Cont....

ISZ: Increment and Skip if Zero

- This instruction increment the word specified by the effective address, and if the incremented value is equal to 0, PC is incremented by 1 to skip the next instruction in the program.
- Since it is not possible to increment a word inside the memory, it is necessary to read the word into DR, increment DR, and store the word back into memory.
- This is done with the following sequence of microoperations:

```
D6T4: DR ← M[AR]  
D6T5: DR ← DR + 1  
D6T6: M[AR] ← DR, if (DR = 0) then (PC ← PC + 1), SC ← 0
```

Cont..

Input-Output and Interrupt:

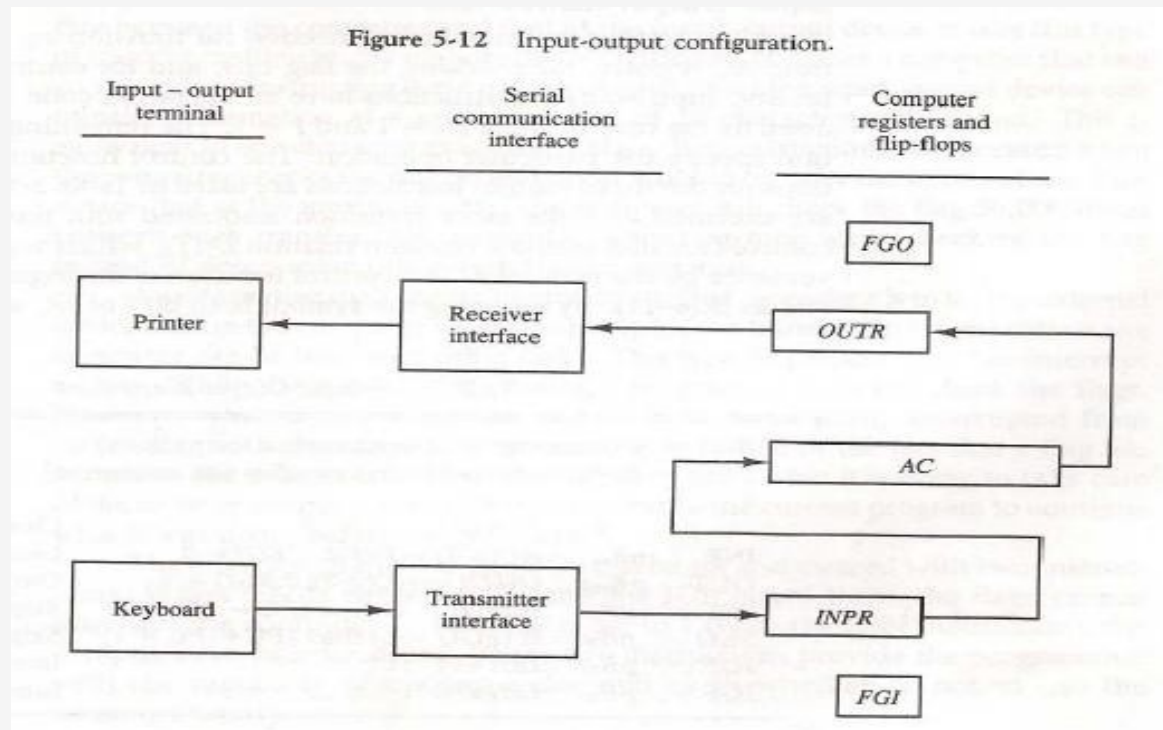
- ❖ Instructions and data stored in memory must come from some input device.
- ❖ Computational results must be transmitted to the user through some output device.
- ❖ To demonstrate the most basic requirements for input and output communication, we will use as an illustration a terminal unit with a keyboard and printer.

Input-Output Configuration:

- ❖ The terminal sends and receives serial information.
- ❖ Each quantity of information has eight bits of an alphanumeric code.
- ❖ The serial information from the keyboard is shifted into the input register INPR.

Cont..

- The serial information for the printer is stored in the output register OUTR.
- These two registers communicate with a communication interface serially and with the AC in parallel.
- The input—output configuration is shown in Fig. 5-12.



Design of Basic Computer

❖ **Basic computer consists the following hardware components**

- A memory unit: 4096 x 16.
- Registers: AR, PC, DR, AC, IR, TR, OUTF, INPR, and SC
- Flip-Flops (Status): I, S, E, R, IEN, FGI, and FGO
- Decoders:
 - a 3x8 Opcode decoder
 - a 4x16 timing decoder
- Common bus: 16 bits
- Control logic gates
- Adder and Logic circuit: Connected to AC

Cont..

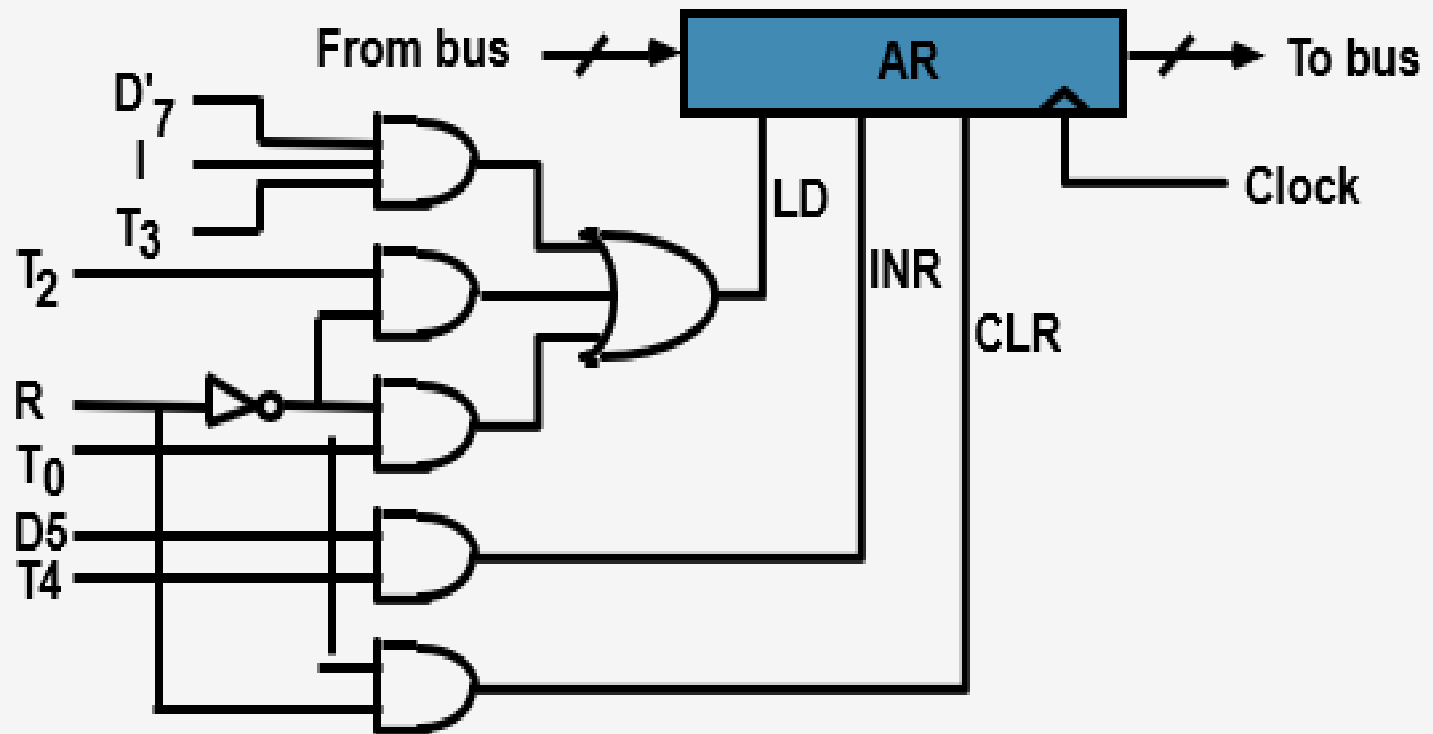
The control logic gates are used to control:

- Inputs of the nine registers
- Read and Write inputs of memory
- Set, Clear, or Complement inputs of the flip-flops
- S₂, S₁, S₀ that select a register for the bus
- AC Adder and Logic circuit

Cont..

- ❖ The control inputs of the registers are LD (load), INR (increment), and CLR (clear)
- ❖ To control AR We scan table 5-6 to find out all the statements that change the content of AR:
 - ✓ R'To: $AR \leftarrow PC$ LD(AR)
 - ✓ R'T2: $AR \leftarrow IR(0-11)$ LD(AR)
 - ✓ D'7IT3: $AR \leftarrow M[AR]$ LD(AR)
 - ✓ RT0: $AR \leftarrow 0$ CLR(AR)
 - ✓ D5T4: $AR \leftarrow AR + 1$ INR(AR)

Cont..



Design of Accumulator logic

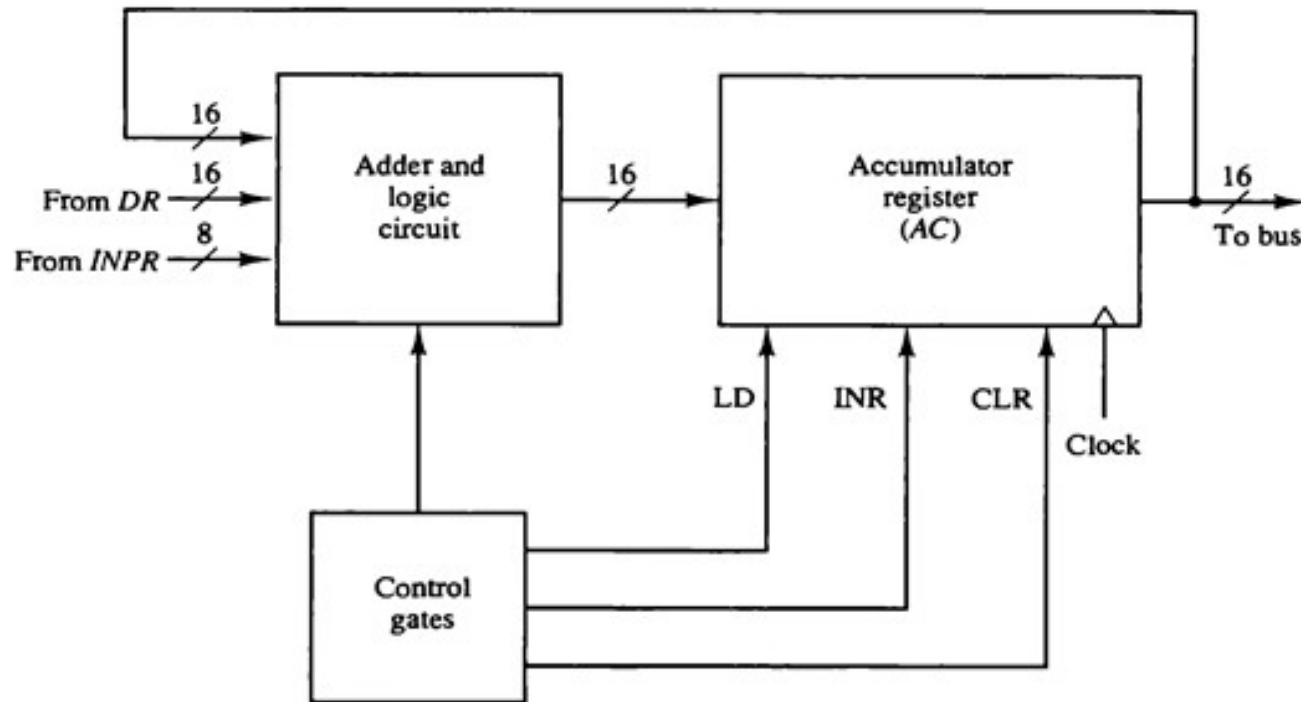
In order to design the logic associated with *AC*, it is necessary to go over the register transfer statements in Table 5-6 and extract all the statements that change the content of *AC*.

$D_0T_5:$	$AC \leftarrow AC \wedge DR$	AND with <i>DR</i>
$D_1T_5:$	$AC \leftarrow AC + DR$	Add with <i>DR</i>
$D_2T_5:$	$AC \leftarrow DR$	Transfer from <i>DR</i>
$pB_{11}:$	$AC(0-7) \leftarrow INPR$	Transfer from <i>INPR</i>
$rB_9:$	$AC \leftarrow \overline{AC}$	Complement
$rB_7:$	$AC \leftarrow \text{shr } AC, \quad AC(15) \leftarrow E$	Shift right
$rB_6:$	$AC \leftarrow \text{shl } AC, \quad AC(0) \leftarrow E$	Shift left
$rB_{11}:$	$AC \leftarrow 0$	Clear
$rB_5:$	$AC \leftarrow AC + 1$	Increment

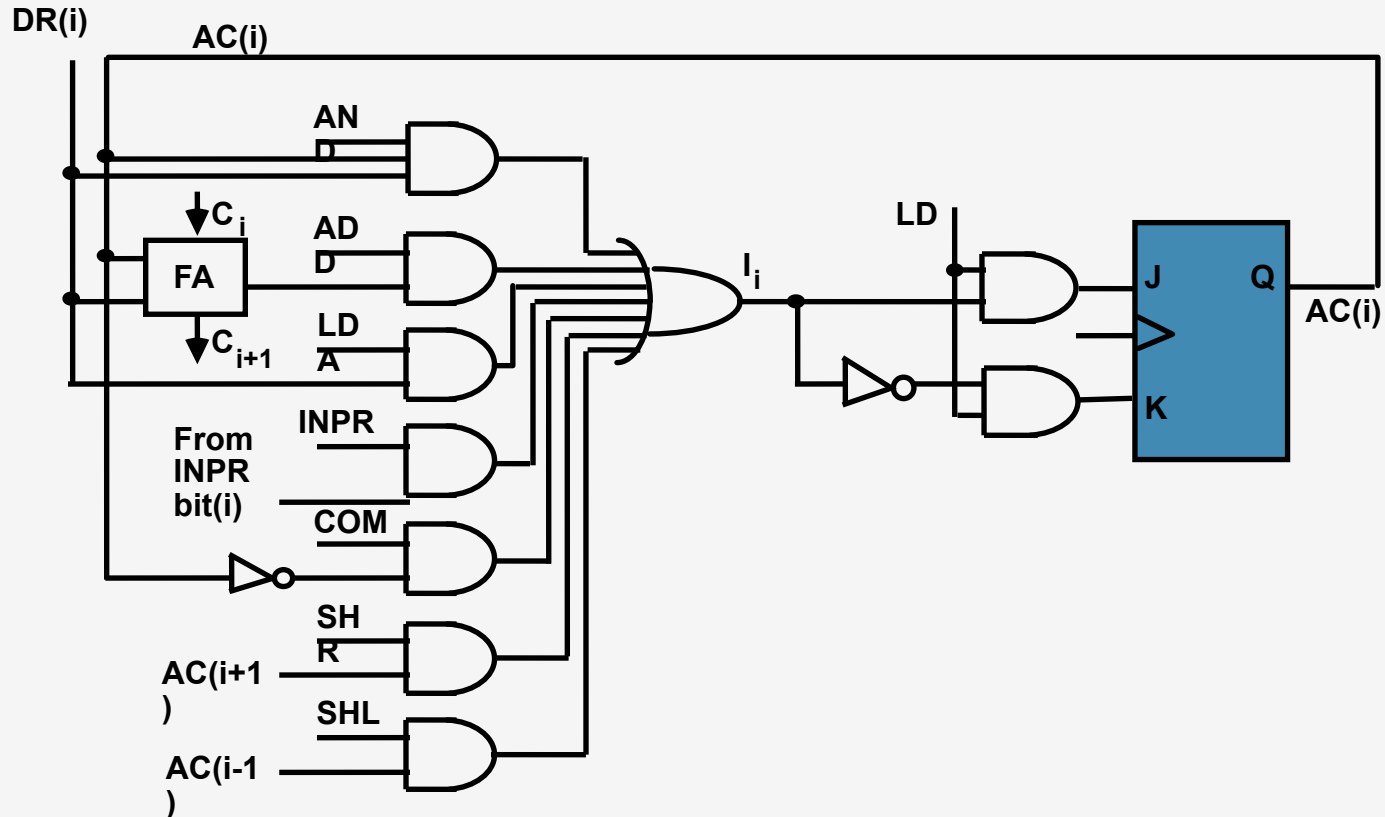
From this list we can derive the control logic gates and the adder and logic circuit.

Design of accumulator logic

Figure 5-19 Circuits associated with AC.



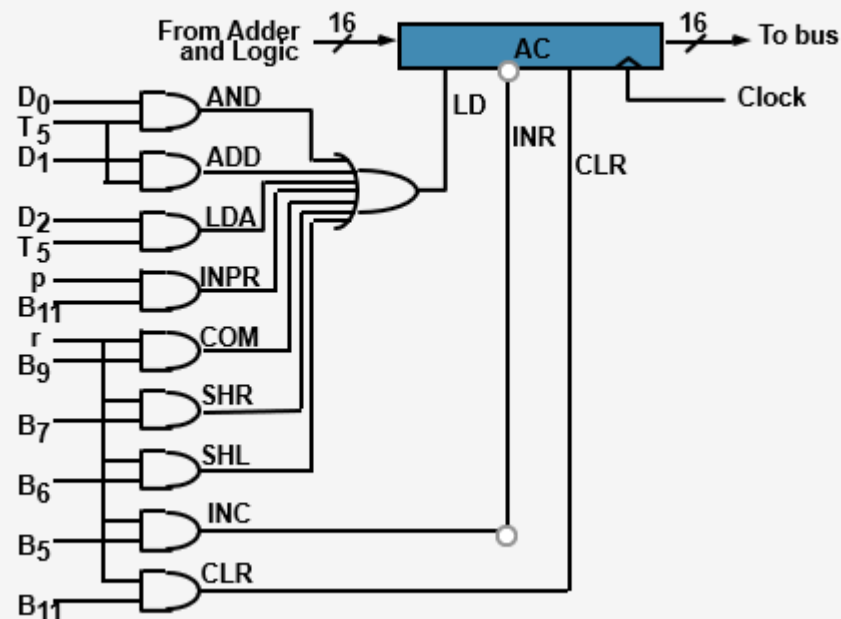
Adder and Logic Circuit



Cont..

- ❖ The control function for the microoperation is rB_n , where
 $r = D_7I'T_3$ and $B_n = IR(11)$.

Gate structures for controlling the LD, INR, and CLR of AC



?